

1.1 First Part:

The first part of the lab consisted on object segmentation through background subtraction. The main idea of this procedure is to detect the foreground objects given a video sequence captured with a static camera. We must choose a reference frame which only contains the background so we can then subtract it from the current frame and apply a threshold to binarize it and therefore set the pixels with an intensity change bigger than the threshold white and keep the rest black.

$$bgs = |curr.pic - ref.pic| > th$$

M1: Following this idea we implemented this naïve approach of the background subtraction algorithm in the function `extract_object`:

```
None
% Subtracting background.
foreground = zeros(rows,columns);
foreground = abs(rgb2gray(input_image/255) -
rgb2gray(image_template/255))*255 > th;
```

For the testing we used 11 consecutive frames where in 5 of them appears a bird flying with a left to right trajectory. We chose the first frame as the reference frame as it didn't contain the object (bird) and we considered a threshold $th = 10$.



Figure 1: Background Picture

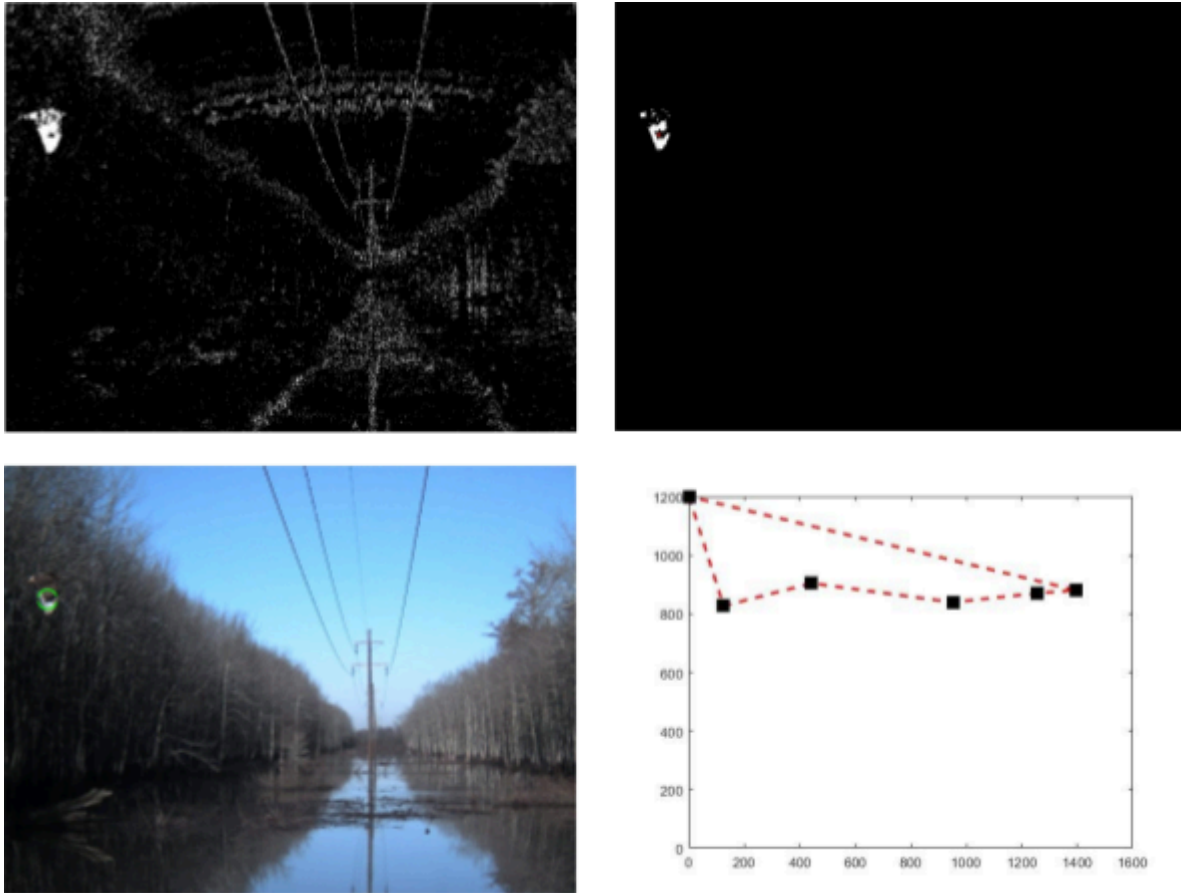


Figure 2: Detected object and trajectory

In Figure2 we can see that there are two binarized images, the first one being noisier than the second one. That is because despite having a static camera, the background is not necessarily static since it can suffer from movements due to external events like wind or illumination changes and also noise can be introduced in the picture. In order to remove this noise from the binarized image we can perform an erosion that will remove those smalls dots (noise) and keep the large regions (foreground object).

M2: To select the optimal threshold, we tested 3 different values $th = [5, 10, 30]$. In these three cases the bird trajectory was well estimated, however the most precise result came from considering a threshold of 10. In the case with $th=5$ some of the pixels of the electricity post survived after the erosion and when considering $th=30$ the bird was severely eroded after the erosion since less part from it survived the threshold. Also, the threshold value is more significant when the difference between the reference and the current frame is higher as it is able to remove those differences that are present due to motion/changes in the background.



Figure 3: Binarization, erode, detection and full trajectory, $th=5$.



Figure 4: Binarization, erode, detection and full trajectory, $th=10$.

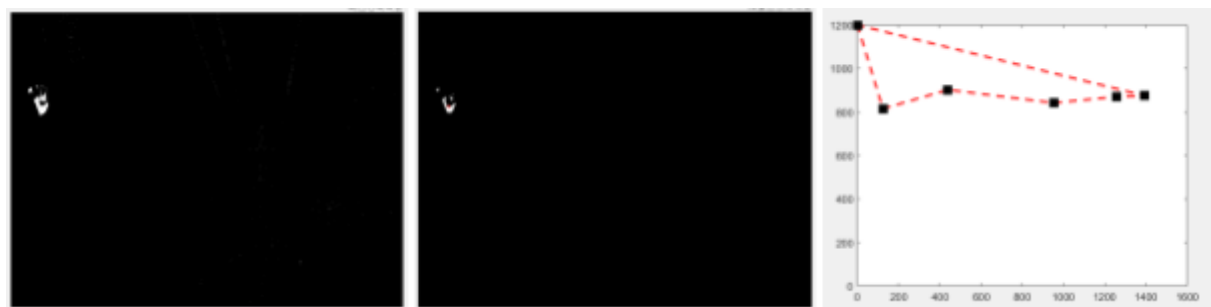


Figure 5: Binarization, erode, detection and full trajectory, $th=30$.

M3: Here we took 10 frames from a video that was also recorded from a static camera. These frames differed quite a lot in illumination and the background had big motion, which caused our naïve approach to fail. It failed because at the beginning we used as a reference picture the first frame and therefore the binarized image even with the erosion was mostly capturing the background rather than the foreground as can be seen in Figure 6. This happened because the first frame had a lot of illumination changes with respect to the other frames.



Figure 6: Bad binarization due to sharp illumination changes

To solve this problem we decided to consider as the reference picture a mean of all the 10 frames.

```
None
% Loading input data
video=VideoReader('input-video.avi');
% Extracting frames (a set of them or, all of them). 10 frames in the example
frames=read(video,[11 20]);
numFrames = size(frames, 4);
mean_frame = zeros(size(frames, 1), size(frames, 2), size(frames, 3));
for f = 1:numFrames
    % imshow(frames(:,:,f));
    % disp('Observing frames. Press any key');
    % pause;
    mean_frame = mean_frame + double(frames(:,:,f));
end
mean_frame = mean_frame / numFrames;
```

This helped to smooth out the sharp illumination changes, the motion of the background and reduced the standard deviation of the noise by $\frac{1}{\sqrt{\sigma}}$ as seen in denoising (assuming noise follows a Gaussian distribution). However this only worked with the frames that were very similar and therefore frames that are consecutive (small variations).

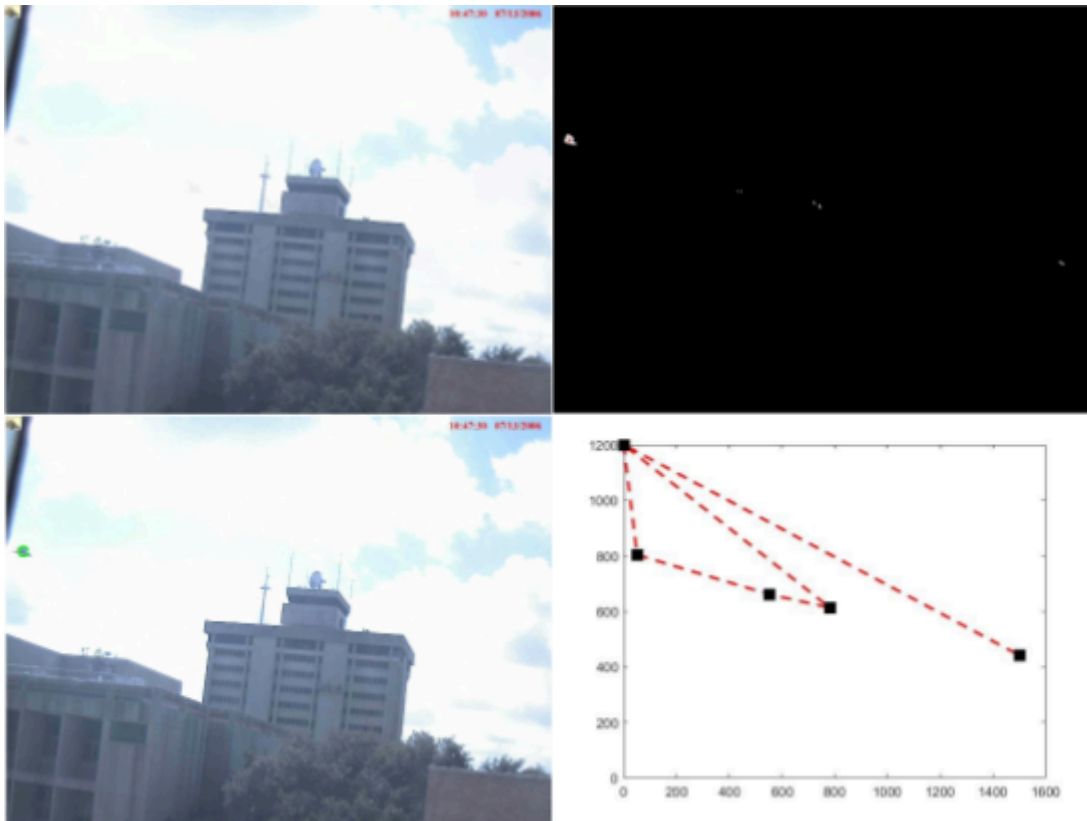


Figure 7: BS taking the mean as reference frame.

Another approach is to use as a reference frame the previous one. This is useful because if we have a continuous video then consecutive frames will be very similar in terms of illumination and background movement.

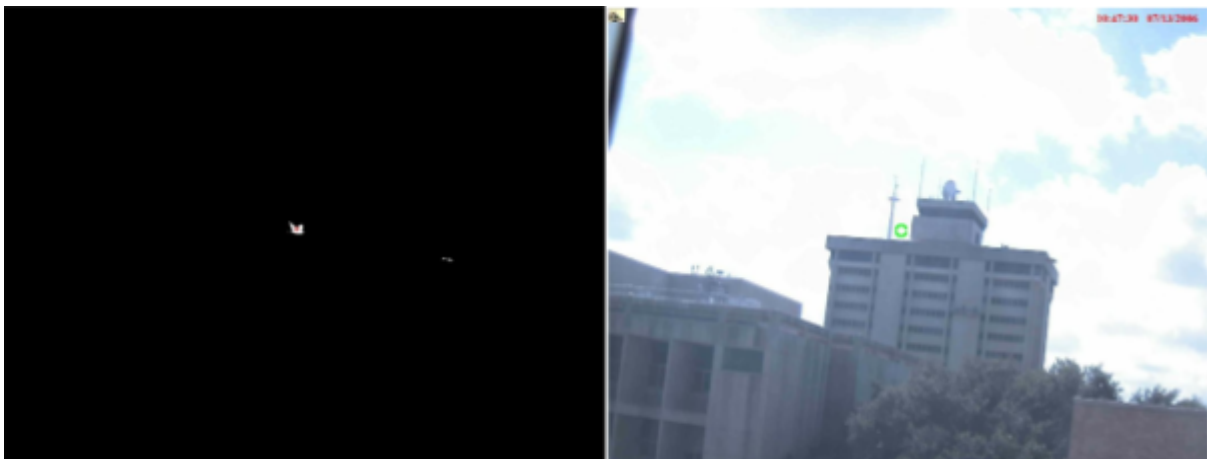


Figure 8: BS taking the previous frame as reference frame.

This fails if the object appears in the previous frame and is bigger than the one appearing in the current frame as seen in Figure 8.

O1: Explain how you could extend your method to detect time-varying shadows.

When considering time varying shadows a good procedure to perform the object segmentation is to consider the anterior frame as the reference frame of each iteration as we just did at the end of the last exercise. As we are now detecting the temporal changes between frames it the model is able to detect illumination changes.

1.2 Second Part:

For this second part, we will perform object segmentation using a graph cut algorithm.

M1: The provided code reads an image (baby or pepper) and divides it into a set number of superpixels (clusters of pixels with similar intensity). Then, it allows us to select 3 bounding boxes: 1 to define the foreground and 2 for selecting the background regions. With this parameters set we can call the Lazzysnapping algorithm, which receives the superpixels and the 3 bounding boxes. This algorithm builds a graph in which every superpixel is a node and connects to the neighbouring superpixels with a weighted edge depending on the intensity (if they have similar intensity the cost of the connection will be high). To detect the foreground objects, the connections with low weight will be cut leaving only the true connected components.

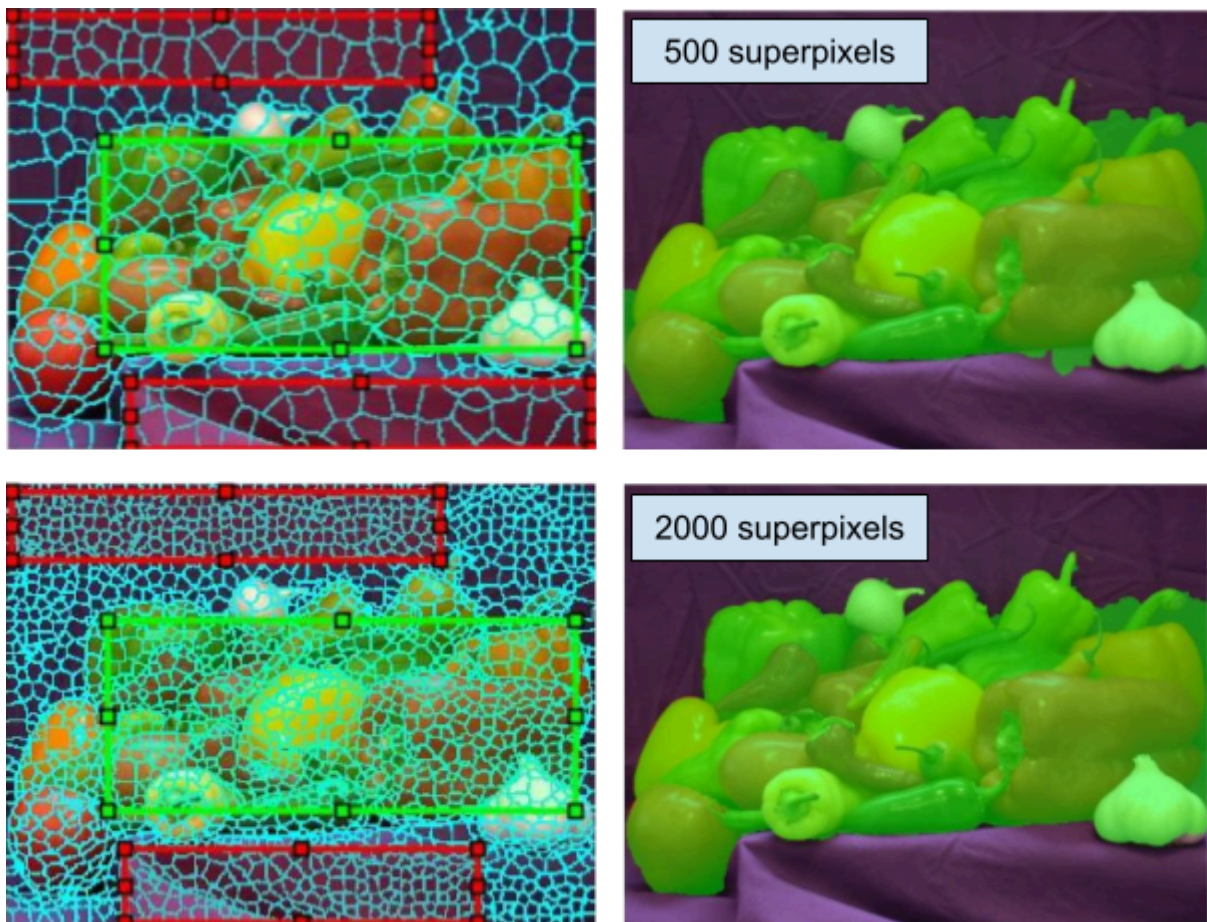


Figure 9: Comparison of graph cut using different amount of superpixels

Having more superpixels makes the segmentation sharper on the edges, but it also increases the computational cost. To reduce the user interaction we removed one background bounding box and the results showed that the segmentation is less accurate since we are reducing the amount of background texture information. Also, if the background is multi-textured then it is more difficult to cover all possible textures (e.g. sky, buildings, etc.).

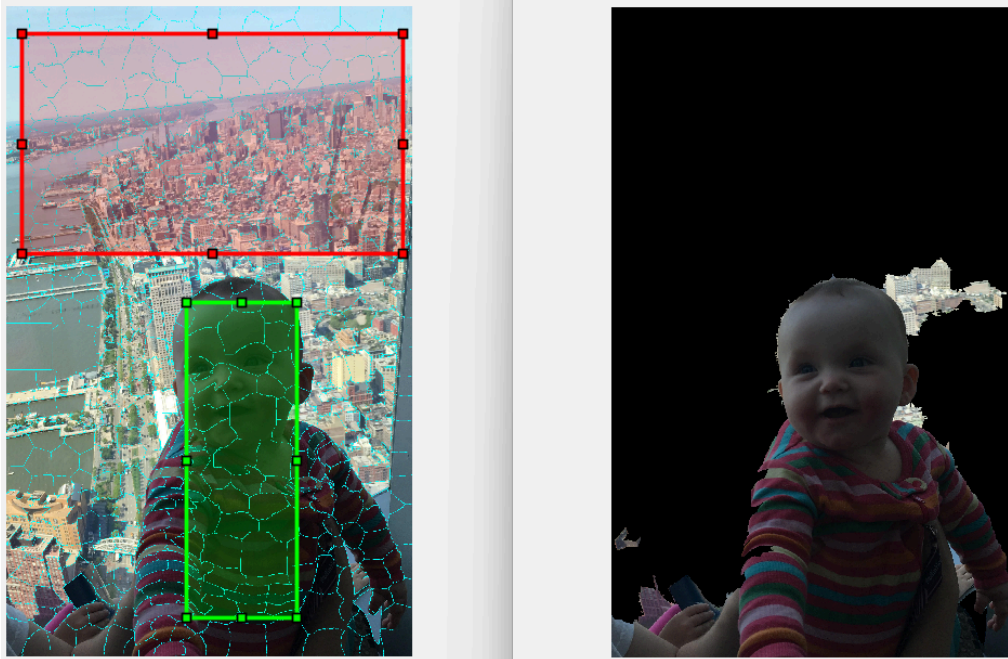


Figure 10: Single Background Bounding Box.

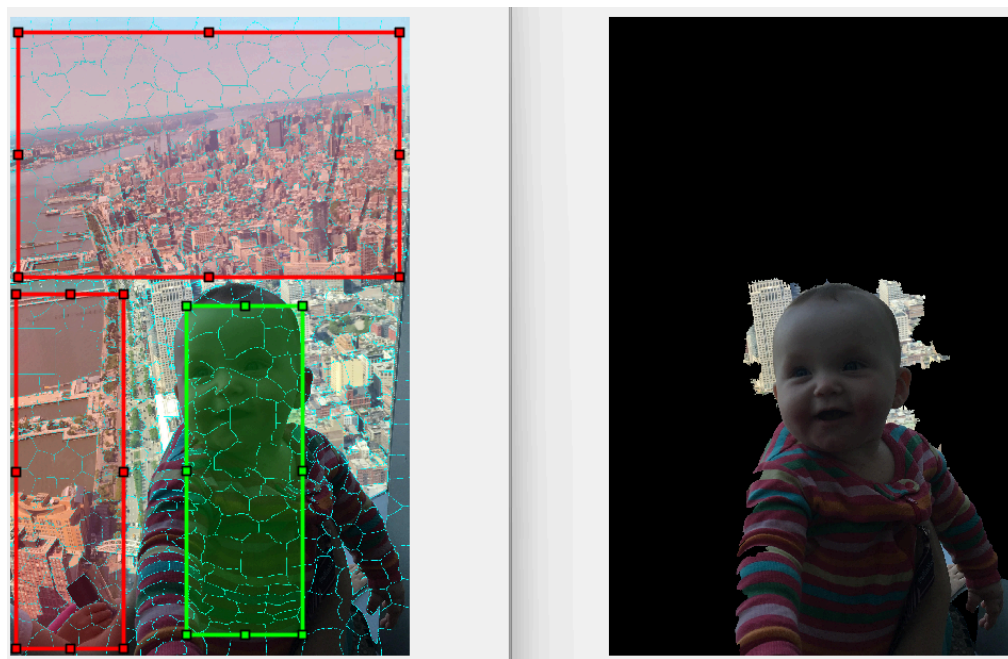


Figure 11: Two Background Bounding Boxes.

M2:

In this section we performed the segmentation by using the matlab tool ImageSegmenter. In this case, the segmentation is done by drawing lines over the region chosen as foreground and the one considered as background. The software computes the mask and automatically estimates the foreground object.

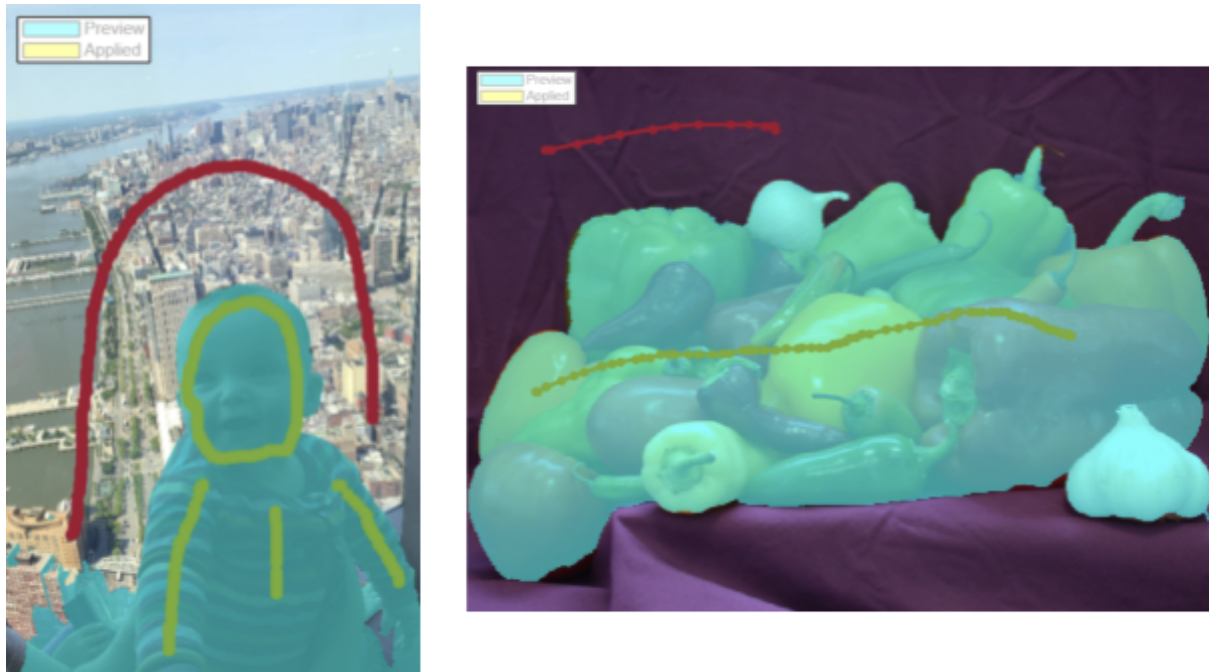


Figure 12: GraphCut using imageSegmenter.

As we can see in Figure 12, this tool with only a few lines provides a really good estimation. The boundaries of the foreground and background are really well defined even in regions with similar color intensities (like the head of the baby and the background) or irregular shapes. As a downfall, we can say that the edges could be sharper because if we look closely we can see how the borders present some irregularities.

M3:

- 1) **Frame-by-frame estimation:** we used three bounding boxes for both the foreground and the background since otherwise it was harder for the algorithm to detect the legs and the neck of the flamingo. This approach basically consists in creating a foreground and background mask using only the annotations (bounding boxes) implemented on the first image and then using the same masks for all subsequent frames, thus reducing user interaction. This works under the assumption that the foreground and background appearance remain relatively constant over time. Finally, we compare each segmentation result with the ground truth to compute each frame's error by doing $1 - accuracy$, where the accuracy is the intersection of the ground truth and our prediction (how many pixels are labeled correctly) divided by the total number of pixels that belong to the ground truth. As we store it in an array we can later one compute the mean error.

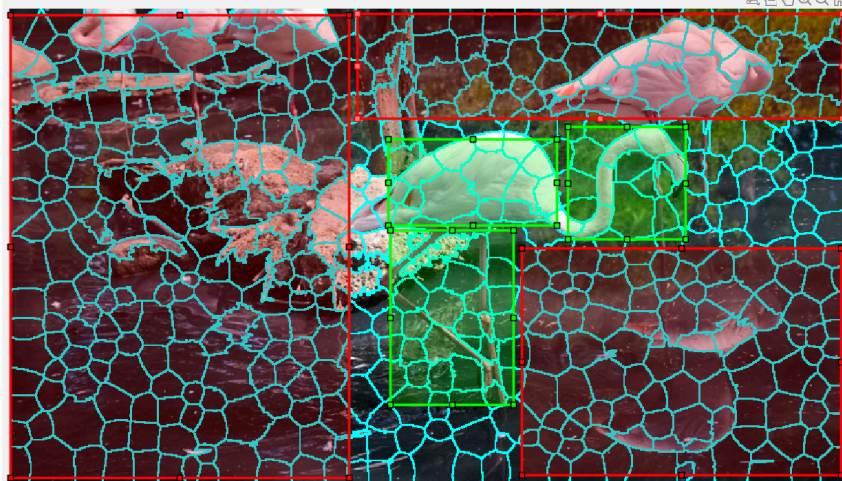


Figure 13: Bounding Boxes.

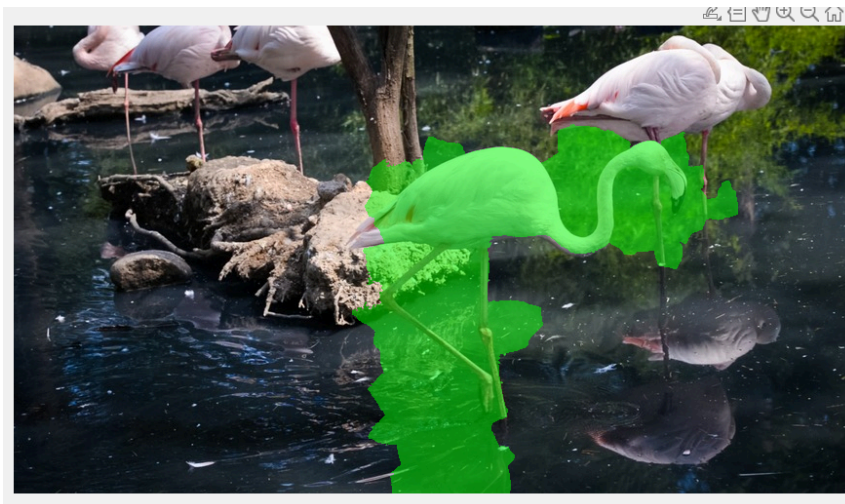


Figure 14: Foreground Estimation of the first frame.

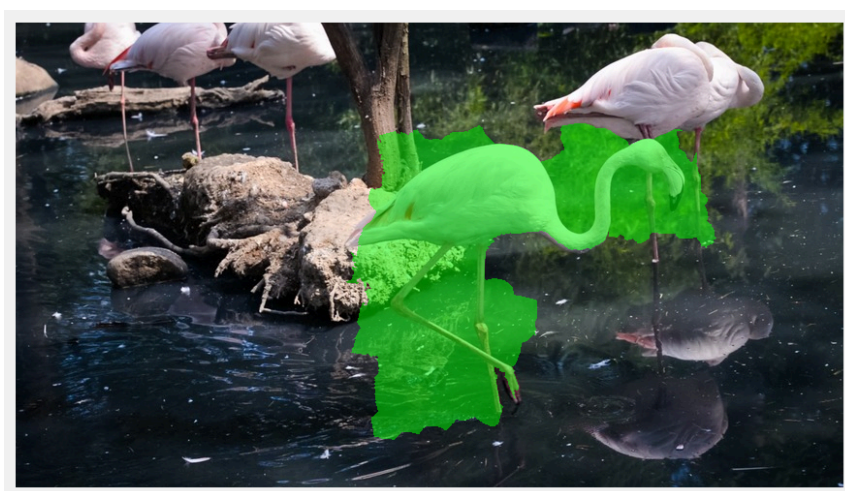


Figure 15: Foreground Estimation of the second frame.

```
Press any key...  
The mean error is 0.0565
```

Figure 16: Mean error comparison with the ground truth.

- 2) Video Estimation: this approach is similar to the previous one but instead of using the foreground and background masks of the first frame, we update them at each iteration so that every frame uses the masks from the previous iteration. Also, we changed the way we create the first mask as we use imageSegmenter so that we can obtain a much accurate estimate.
- When executing the code we first have to create a mask using imageSegmenter and export it to the workspace as BW1 so that it does not crash:



Figure 17: Foreground and Background segmentation using imageSegmenter.

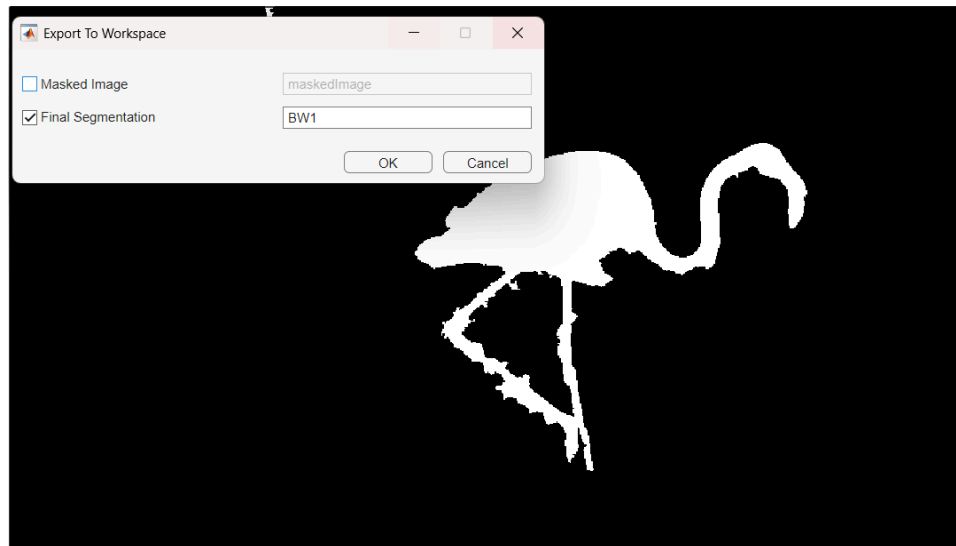


Figure 18: Final mask and export to workspace.

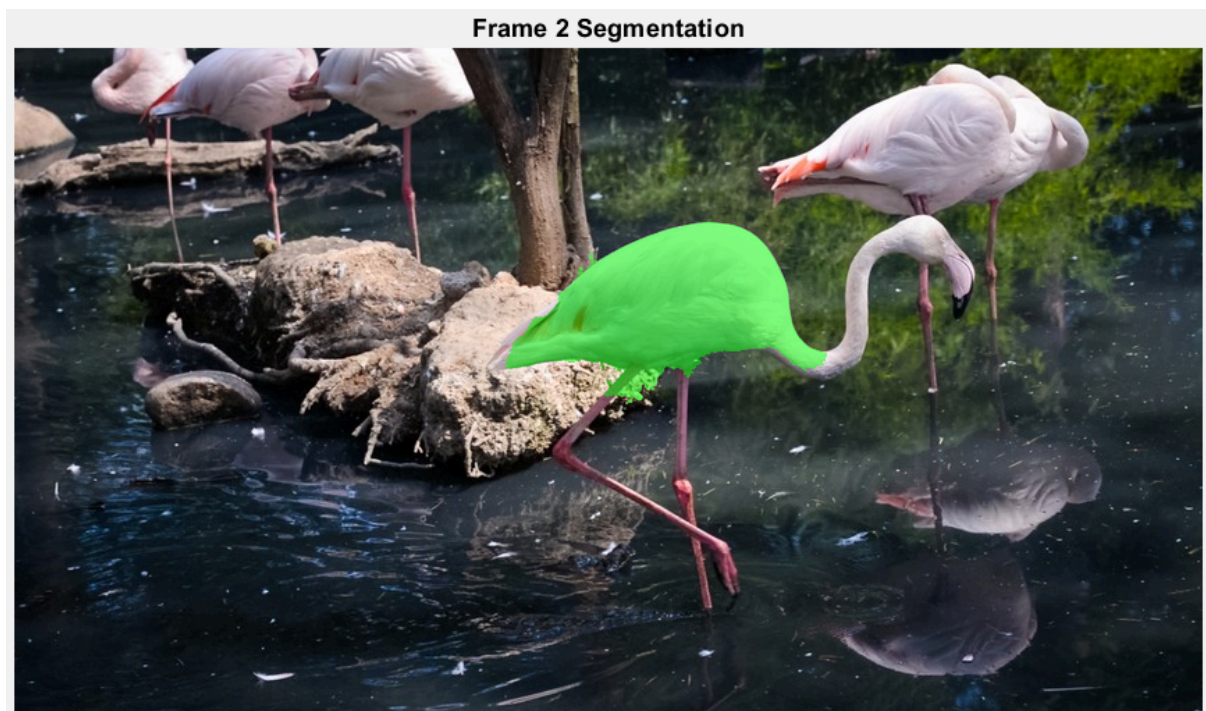


Figure 19: Segmentation of the second frame using previous mask.

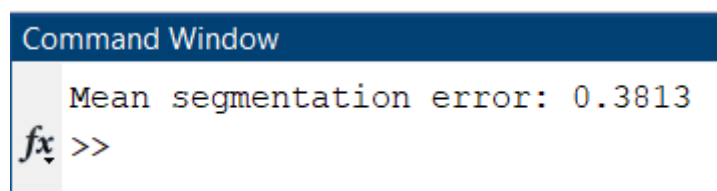


Figure 20: Mean segmentation error.

1.3 Third Part:

M1: This matlab code implements scene segmentation using subspace clustering via Ordered Subspace Clustering (OSC) and Normalized Cuts (NCut). It simulates video frames that belong to different scenes, where each frame consists in $dim_data = 321$ (pixels, features, etc), there are $n_space = 5$ clusters, which are the number of scenes and each scene has 10 frames so $cluster_size = 10$. After applying some noise, it computes the affinity matrix with the function `osc_relaxed`. This matrix Z is symmetric and each column of it represents how one frame is expressed as a linear combination of another frame (row). Therefore, the larger the values, the stronger the relation and similarity between those two frames. This is represented by a yellowish color. For example, if we take a first order filter with the highest corruption value (0.18) we obtain a high error of the clusterization.

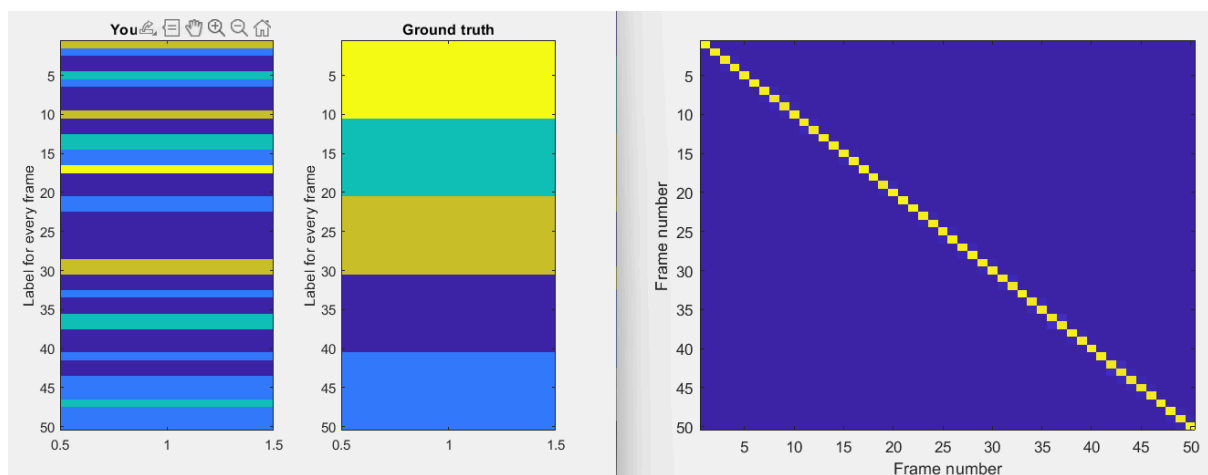


Figure 21: Frame labeling and Z (1st order filter & corruption = 0.18)

As we can see the error is high because the affinity matrix is a diagonal matrix, which means that each frame is only strongly related to itself.

M2: In this section we had to complete the function `osc_relaxed` with the implementation of a second order and a fourth order regularization filters R . As seen in theory the matrices that characterize have the following shape:

$$\mathbf{R}_2 = \begin{bmatrix} 1 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & -1 & \\ & & \ddots & \ddots & \ddots \\ & & & -1 & 1 \end{bmatrix} \quad \mathbf{R}_4 = \begin{bmatrix} 0 & 1 & -1 & & & & \\ 1 & -16 & 16 & -1 & & & \\ -1 & 16 & -30 & 16 & -1 & & \\ & -1 & 16 & -30 & 16 & -1 & \\ & & -1 & 16 & -30 & 16 & -1 \\ & & & \ddots & \ddots & \ddots & \ddots \\ & & & & -1 & 1 & 0 \end{bmatrix}$$

Figure 22: Second and fourth order

In matlab we implemented this matrices by using a combination of `triu` matrices (which are matrices that only have values above a certain diagonal) scaled to obtain the desired indices.

M3: Once we had the 3 filters implemented we computed the following error graphs by iterating over all the corruption levels in all 3 cases and saving the error in an histogram at each iteration. The results showed that the second order filter performed the best results.

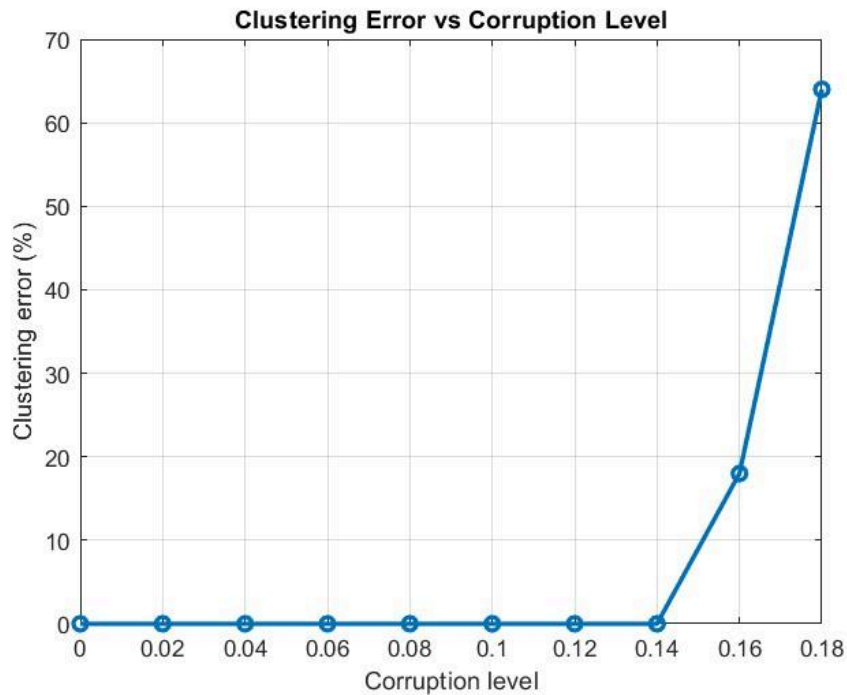


Figure 23: Clustering error vs. corruption with a 1st order filter.

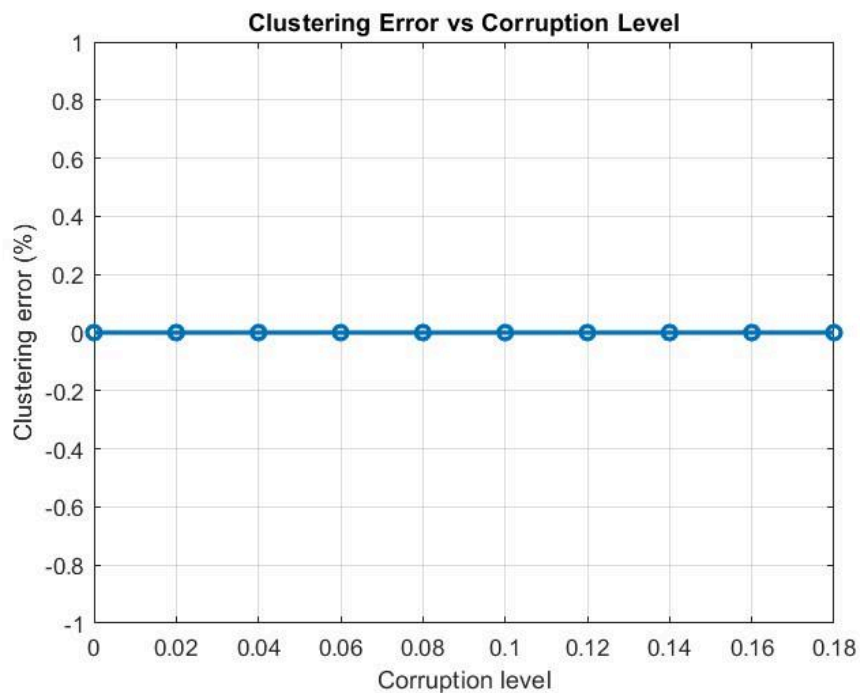


Figure 24: Clustering error vs. corruption with a 2nd order filter.

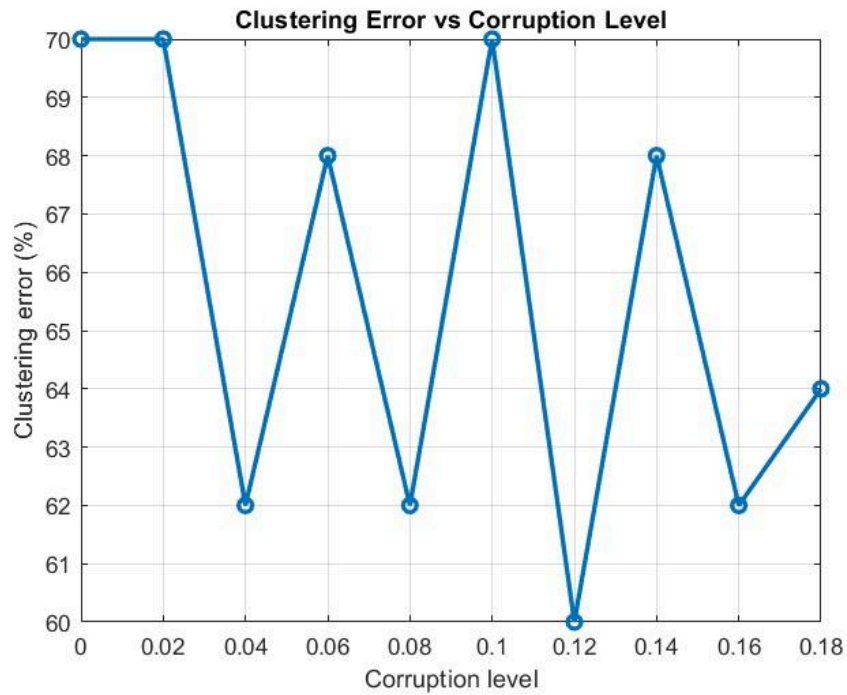


Figure 25: Clustering error vs. corruption with a 4th order filter.

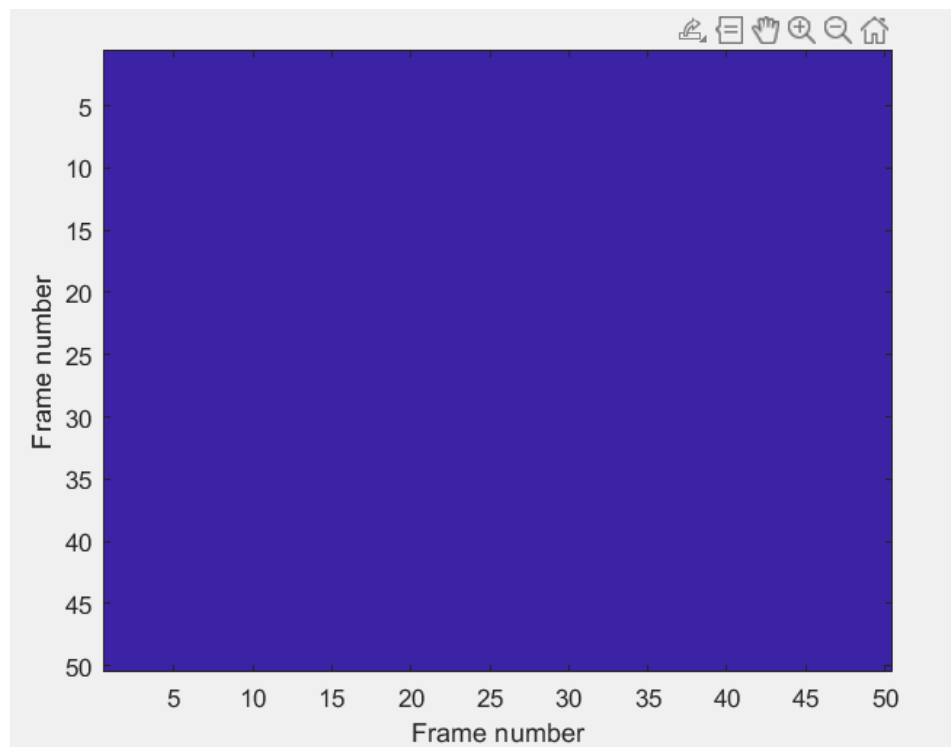


Figure 26: Affinity matrix Z with a 4th order filter.

In Figure 26 we can see that the affinity matrix becomes uniform with no connections at all between frames.

M4:

The OSC (Ordered Subspace Clustering) algorithm solves a sparse representation problem with temporal consistency and the filter controls the temporal smoothing order. Therefore, the 1st order filter only considers the immediate previous frame, the 2nd order filter considers the last 2 frames, and the 4th order filter considers the last 4 frames. Hence, larger filter enforces longer temporal correlations.

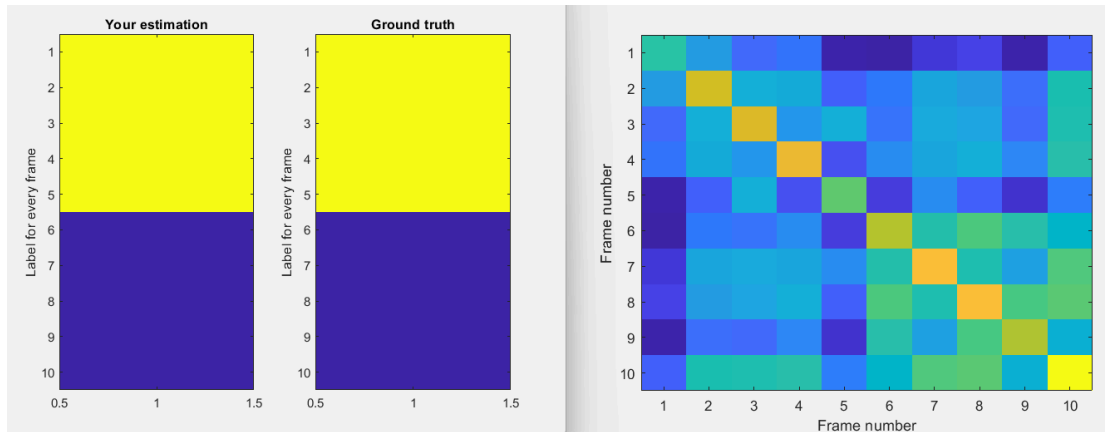


Figure 27: First order filter result

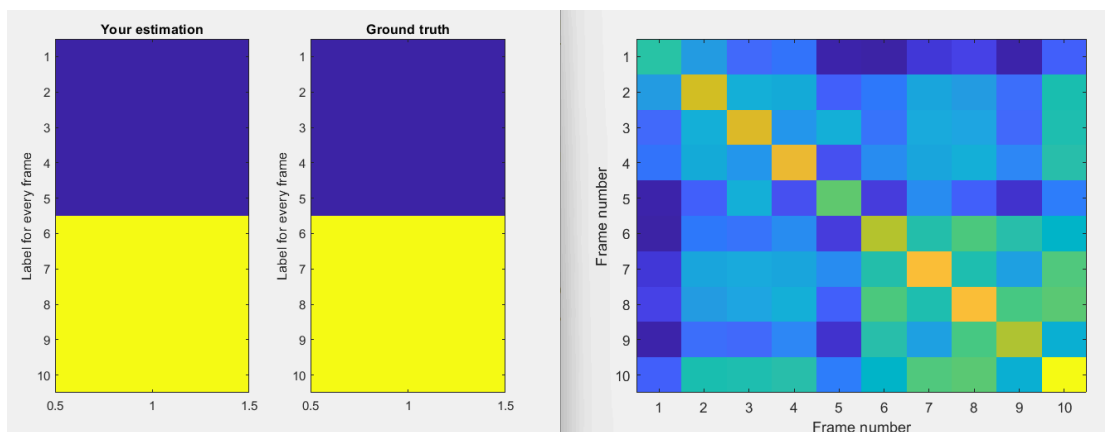


Figure 28: Second order filter result

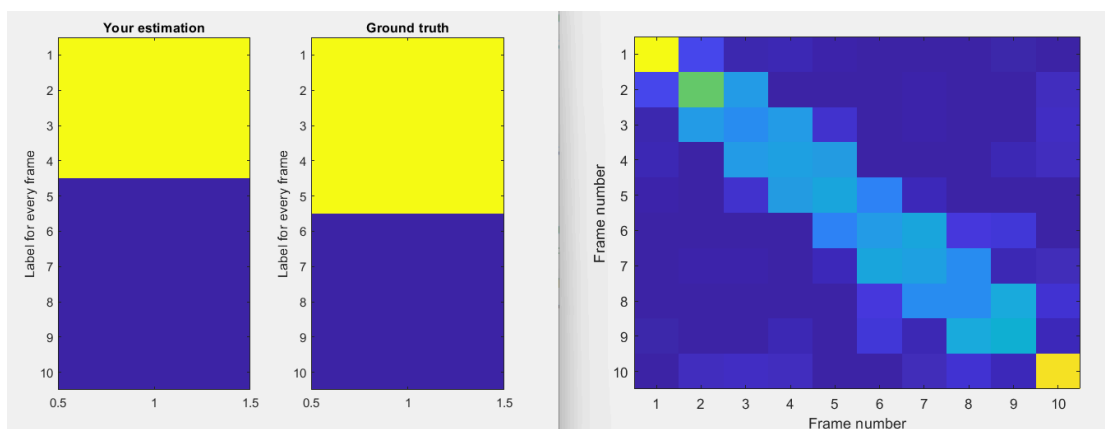


Figure 29: Fourth order filter result

Higher-order filters increase temporal smoothing, which is good for continuous scenes. However, if we have abrupt scene changes, such as the sharp transition between a frame containing the number 1 and another frame containing number 7, a large filter blurs the cluster boundary, introducing misclassification, which results in a high error when the number of frames per cluster is small. That's exactly what we can notice at Figure 29 since a 4th order filter is misclassifying the frame at the boundary between scenes.

1.4 Fourth Part:

M1:

A pixel-by-pixel annotation is a labeling method where every pixel of an image is assigned to a class/category label. To assign a label to an object we created polygons that surrounded that instance.

We didn't assign a different label to each background object, but instead we created a general background label to simplify it (same with ring ropes).

- 1) Semantic Segmentation: each pixel is assigned to a semantic class that can correspond to a real-world object. For example, in the boxing dataset in each frame we could identify two boxers, a referee, a ring, spectators, and other background objects. Therefore, the pixels within the boxers will be assigned the same label even though they belong to different instances.

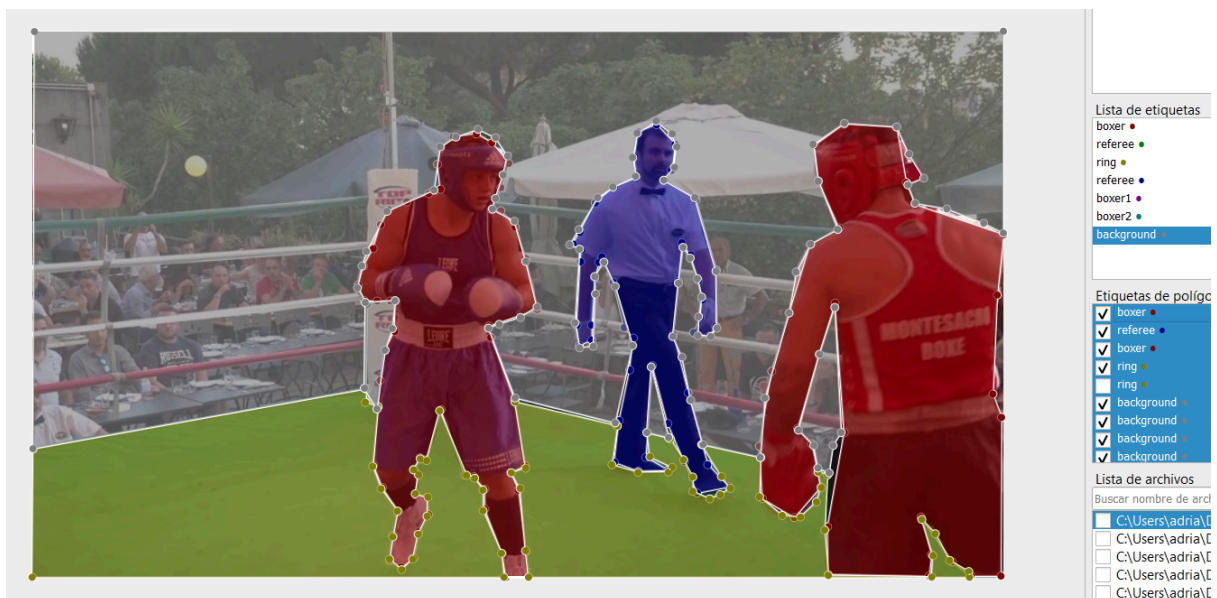


Figure 30: Manual semantic Segmentation.

As we can see in Figure 30, both boxers have the same color (label), while the referee and the floor of the ring have different colors.

- 2) Instance Segmentation: each pixel in the image is assigned both a class label and an object identity. Unlike semantic segmentation, it distinguishes different objects of the same class (e.g., boxer1 and boxer2 instead of just “boxer”).

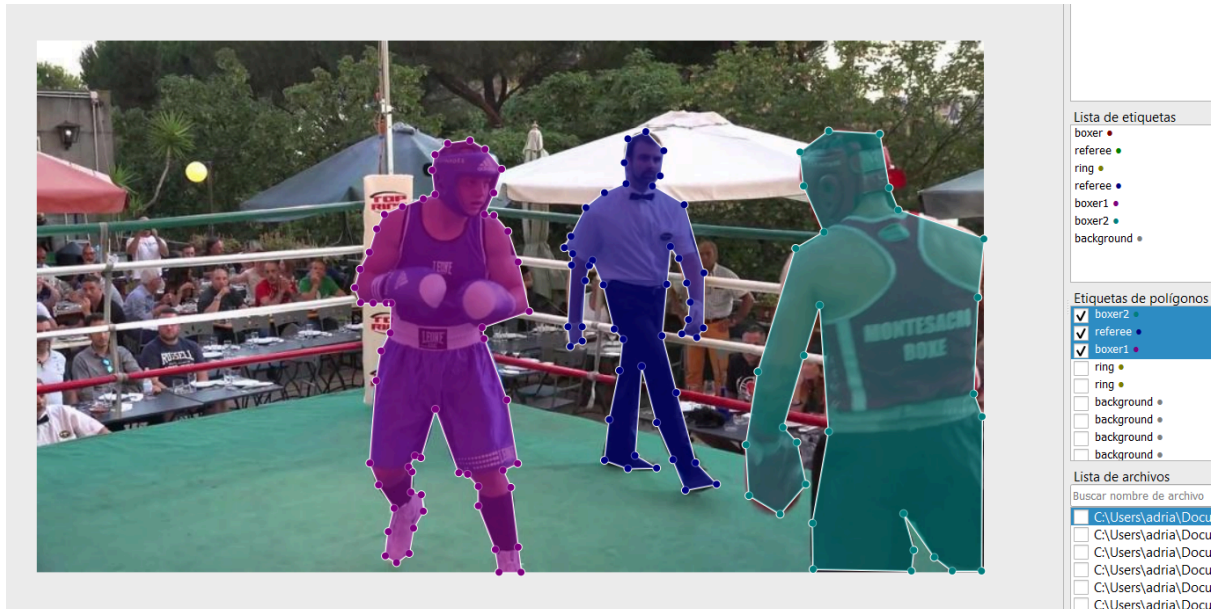


Figure 31: Manual instance Segmentation.

These annotations, for the context of boxing events, could be used to train a semi-supervised video object segmentation. to perform online/offline (depending if it is a livestream or a recording) generation of performance statistics, such as punch detection, distance estimation between fighters, knockout counting or to identify referee interventions. Another useful application could be applying labeling for an automatic camera tracking software so that it automatically follows the boxers.

M2:

Panoptic Segmentation: combines the prediction from both semantic and instance segmentation to create a labeling solution where some particular objects are treated individually (as instances), while others belong to the same class (semantic). Therefore, we can use the segmentations done to solve a panoptic image segmentation problem because we can consider the three different instances (two boxers and the referee) as the “objects” and then the rest of the image as the “stuff” (contained in the same label) . However, for extending it to video, we should consider that we need to segment objects through occlusions, illumination variations and maintain temporal consistency. To address this, we could either annotate each frame individually (fully supervised) or annotate only the first frame and propagate the segmentation using object tracking methods (semi-supervised).

M3:

To perform the three segmentations with SAM, we first opened the online editor and clicked at the boxers, the referee and the ring floor. The results of these three segmentations were really accurate. We tried to get wrong segmentations with the online editor but it always performed good.

Additionally we did the same three object segmentations with the SAM included inside the LabelMe application. Using the AI-polygon option, we clicked at the boxers and the referee and obtained some well estimated polygons. However, to get the correct results we had to click at several points spaced along the boxer (or referee) because if not the model would only take part of the object. This failure cases can be seen in the Figure 34 in which for example the boxer has been segmented without the gloves, and for the referee the model has taken only the shirt as an object.

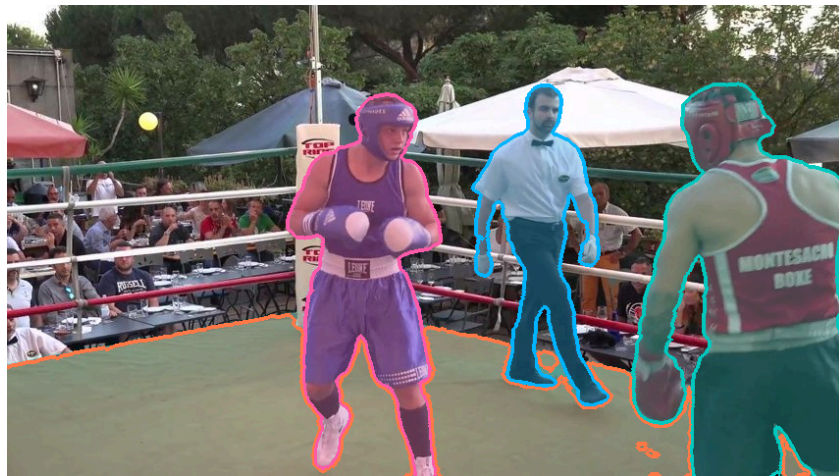


Figure 32: segmentation with SAM online tool.



Figure 33: segmentation with SAM inside labelmel.



Figure 34: wrong segmentation with SAM inside labelmel.